

Spring Framework – Day 21

Annotations (Day 04): @Autowired – Setter, Constructor, Field and Method Injection, Ambiguity Resolution with @Qualifier

1. Setter Injection: Two Dependencies with @Autowired

When a class has two dependencies and both need to be injected by the IOC container, **@Autowired** can be placed on each setter method. Spring resolves each dependency by type (byType) and calls the matching setter.

Student.java

```
public class Student {
    Course course;
    Institute institute;

    public Student() {
        System.out.println("Student class object");
    }

    @Autowired
    public void setCourse(Course course) {
        System.out.println("Through student setter of course");
        this.course = course;
    }

    @Autowired
    public void setInstitute(Institute institute) {
        System.out.println("Through student setter of institute");
        this.institute = institute;
    }

    @Override
    public String toString() {
        return "Student [course=" + course + ", institute=" + institute + "];"
    }
}
```

application-context.xml

```

<beans>

    <bean id="course" class="com.day20.Course">
        <property name="name" value="spring framework"></property>
    </bean>

    <bean id="institute" class="com.day20.Institute">
        <property name="name" value="NIT"></property>
    </bean>

    <bean id="student" class="com.day20.Student">
        <!-- <property name="course" ref="course"></property> -->
    </bean>

    <context:annotation-config></context:annotation-config>

</beans>

```

OUTPUT

```

Student [course=Course [name=spring framework],
        institute=Institute [name=NIT]]

```

1.1 What if @Autowired is removed from the institute setter?

The course setter still has @Autowired, so course is injected as before. The institute setter no longer has @Autowired, so Spring never calls it, and institute stays at its default value.

OUTPUT

```

Student [course=Course [name=spring framework], institute=null]

```

1.2 What if @Autowired is removed from both setters?

Neither setter is called by the IOC container, so both dependencies remain unset.

OUTPUT

```

Student [course=null, institute=null]

```

1.3 What if the bean definition is removed from the XML, but @Autowired is still present on both setters?

The Student object is still created. However, @Autowired is **required = true** by default, which makes setter injection mandatory. Since the matching bean (course or institute) is not available in the configuration, Spring cannot satisfy the mandatory dependency and throws an exception.

KEY POINT

By default, @Autowired means required = true. If Spring cannot find a matching bean for a mandatory dependency, it throws an exception instead of leaving the field as null.

2. Constructor Injection with Multiple @Autowired Constructors

A class can have more than one constructor, and @Autowired can be placed on more than one of them. Here we look at what happens when a class has two dependencies and two separate single-argument constructors, each annotated with @Autowired.

Student.java

```

public class Student {
    Course course;
    Institute institute;

    public Student() {
        System.out.println("Student class object");
    }

    @Autowired
    public Student(Course course) {
        super();
        System.out.println("Through student constructor");
        this.course = course;
    }

    @Autowired
    public Student(Institute institute) {
        super();
        this.institute = institute;
    }

    // toString()
}

```

application-context.xml

```

<beans>

    <bean id="course" class="com.day20.Course">
        <property name="name" value="spring framework"></property>
    </bean>

    <bean id="institute" class="com.day20.Institute">
        <property name="name" value="NIT"></property>
    </bean>

    <bean id="student" class="com.day20.Student">
        <!-- <property name="course" ref="course"></property> -->
    </bean>

    <context:annotation-config></context:annotation-config>

</beans>

```

OUTPUT

```
Invalid autowire-marked constructor
```

By default, `@Autowired` on a constructor means `required = true`. Since both constructors are marked `required = true`, both become mandatory — but the IOC container can call only one constructor while creating an object, so it cannot satisfy both. This is not possible, and Spring reports an error.

2.1 What if one constructor is `required = true` and the other is `required = false`?

```

@Autowired
public Student(Course course) {
    super();
    System.out.println("Through student constructor");
    this.course = course;
}

@Autowired(required = false)
public Student(Institute institute) {
    super();
    this.institute = institute;
}

```

This is not a recommended approach. When one constructor is mandatory (`required = true`), the container always uses it, so the optional (`required = false`) constructor never gets called anyway. There is little value in writing a second constructor that Spring will never choose to run.

2.2 Can `@Autowired` be placed on multiple constructors?

Yes — if **every** constructor is marked `@Autowired(required = false)`.

In this case, the IOC container cannot decide on its own which constructor must be called, since all of them are optional. From Spring 6.0.x onward, the container simply injects one of them:

OUTPUT

```
Student [course=null, institute=Institute [name=NIT]]
```

VERSION DIFFERENCE

In older versions (before Spring 5.0.x), this same situation resulted in an exception instead of Spring silently picking one constructor.

2.3 What if the class has a two-parameter constructor and a one-parameter constructor?

```

@Autowired(required = false)
public Student(Course course, Institute institute) {
    super();
    System.out.println("Through student constructor");
    this.course = course;
    this.institute = institute;
}

@Autowired(required = false)
public Student(Institute institute) {
    super();
    System.out.println("Through institute constructor");
    this.institute = institute;
}

```

There is no exception. Spring injects the dependencies using the constructor that takes the maximum number of parameters that it is able to satisfy.

OUTPUT

```
No exception – injected through the constructor with the maximum number of parameters.
```

3. Field-Level @Autowired

@Autowired can also be placed directly on a field (an attribute), without writing any constructor or setter method for it.

Student.java

```
public class Student {  
  
    @Autowired  
    Course course;  
  
    @Autowired  
    Institute institute;  
  
    @Override  
    public String toString() {  
        return "Student [course=" + course + ", institute=" + institute + "];"  
    }  
}
```

With all the bean definitions present in the configuration file, the output is:

OUTPUT

```
Student [course=Course [name=spring framework],  
        institute=Institute [name=NIT]]
```

How does this work?

The IOC container uses the Reflection API to inject the dependency directly into the field of the target class — even though the field is declared as private.

Constructor vs. setter vs. field injection — which one to choose?

If a class already supports both constructor injection and setter injection, when should field-level @Autowired be used instead? Consider the following two class designs.

Case A – constructor injection, no post-construction logic

```
class A {  
    B b;  
  
    A(B b) {  
        this.b = b;  
    }  
}  
  
class B {  
}
```

Here, class A takes B through the constructor purely to store the reference, with no extra logic performed after construction. Since there is no real post-construction work happening in the constructor, writing a full constructor just for this purpose is not necessary. In such cases, it is better to use setter injection instead.

Case B – setter injection

```

class A {
    B b;

    public void setB(B b) {
        this.b = b;
    }
}

class B {
}

```

If immutability is important (the dependency should not be changed after the object is created), a setter should be avoided as well, since a setter allows the value to be modified later. In that situation, field or constructor injection is preferred over a setter.

4. @Autowired on an Arbitrary Method

@Autowired is not limited to constructors, setters, or fields. It can also be placed on any method of the class (an "arbitrary" method), and Spring will call that method to inject the dependencies passed as its parameters.

Student.java

```

public class Student {

    private Course course;
    private Institute institute;

    @Autowired
    public void student(Course course, Institute institute) {
        this.course = course;
        this.institute = institute;
    }

    @Override
    public String toString() {
        return "Student [course=" + course + ", institute=" + institute + "]";
    }
}

```

5. Resolving Ambiguity with Multiple Beans of the Same Type

When more than one bean of the same type is present in the configuration file, which one should the IOC container inject? This ambiguity issue occurs only when resolution happens byType.

Previously (in XML-based configuration), this ambiguity was solved by setting **autowire-candidate="false"** on the bean that should be excluded from autowiring.

@Autowired also identifies beans byType, so the same ambiguity problem applies here. Without XML configuration, this is resolved using **@Qualifier**. By default, the bean id is treated as the qualifier name.

Student.java

```

public class Student {

    private Course course;
    private Institute institute;

    @Autowired
    @Qualifier("course1")
    public void setCourse(Course course) {
        this.course = course;
    }

    @Autowired
    public void setInstitute(Institute institute) {
        this.institute = institute;
    }

    @Override
    public String toString() {
        return "Student [course=" + course + ", institute=" + institute + "]";
    }
}

```

application-context.xml

```

<beans>

    <bean id="course" class="com.day20.Course">
        <property name="name" value="spring framework"></property>
    </bean>

    <bean id="course1" class="com.day20.Course">
        <property name="name" value="Java"></property>
    </bean>

    <bean id="institute" class="com.day20.Institute">
        <property name="name" value="NIT"></property>
    </bean>

    <bean id="student" class="com.day20.Student">
    </bean>

    <context:annotation-config></context:annotation-config>

</beans>

```

KEY POINT

There are two beans of type Course (course and course1), which would otherwise be ambiguous for byType autowiring. @Qualifier("course1") tells Spring exactly which bean to inject into the course field, resolving the ambiguity.